

© Akshat Paul and Abhishek Nalwaya 2019

Akshat Paul and Abhishek Nalwaya, *React Native for Mobile Development*

https://doi-org.ezproxy.sfpl.org/10.1007/978-1-4842-4454-8_3

3. Solving Problems Differently with Flux and Redux

Akshat Paul¹ and Abhishek Nalwaya²

(1) Gurgaon, Haryana, India

(2) Jaipur, Rajasthan, India

Simplicity is prerequisite for reliability.

—Dijkstra

Flux is an application architecture introduced by Facebook for building client-side applications. It complements the React paradigm of composable view components by using a unidirectional data flow. It's more of a pattern than a framework, and one can start using Flux immediately without an excess load of code. Redux is a predictable state container for JavaScript applications, which means it helps us to write applications that behave consistently in different environments: client, server, or native. It also makes your applications easy to debug and test.

Before we delve into its details, it is important to know one of the most popular, commonly used MVC patterns. We can then learn about what challenges we face with MVC and how Flux and Redux can solve these challenges.

This chapter covers the following topics:

- MVC pattern
- MVC problem
- Flux
- Flux deep dive
- Redux
- Redux core concepts
- Redux with React Native
- Benefits of using Redux

MVC Pattern

Historically, an MVC pattern separates code into three distinct parts: model, view, and controller. The main purpose of this pattern is to isolate representation of information from user interaction. Let's describe each of these parts individually.

- *Model*: This element manages the behavior and data of an application.
- *View*: This is the representation layer of the model in the UI.
- *Controller*: This element takes user input and makes necessary manipulations to the model, which causes the view layer to get updated.

MVC is legendary and it's an amazing way to structure your code. Things get a bit ugly, though, when your source code begins to grow and get complex. Although MVC is a very popular pattern to design applications, it comes with its own set of problems. Figure [3-1](#) shows how MVC works.

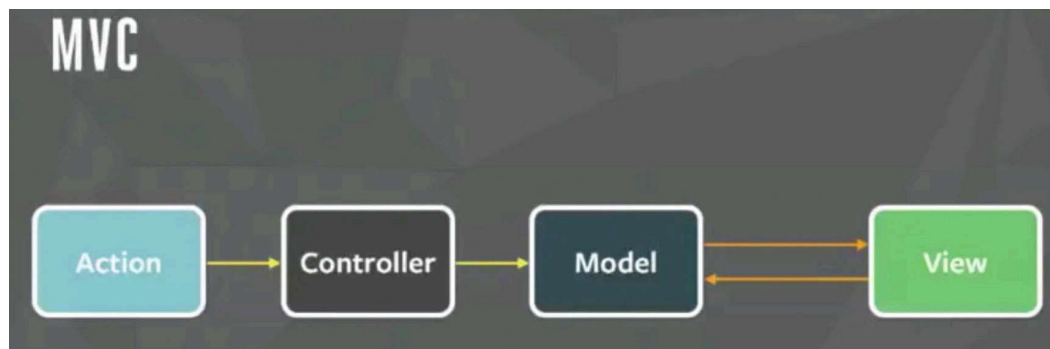
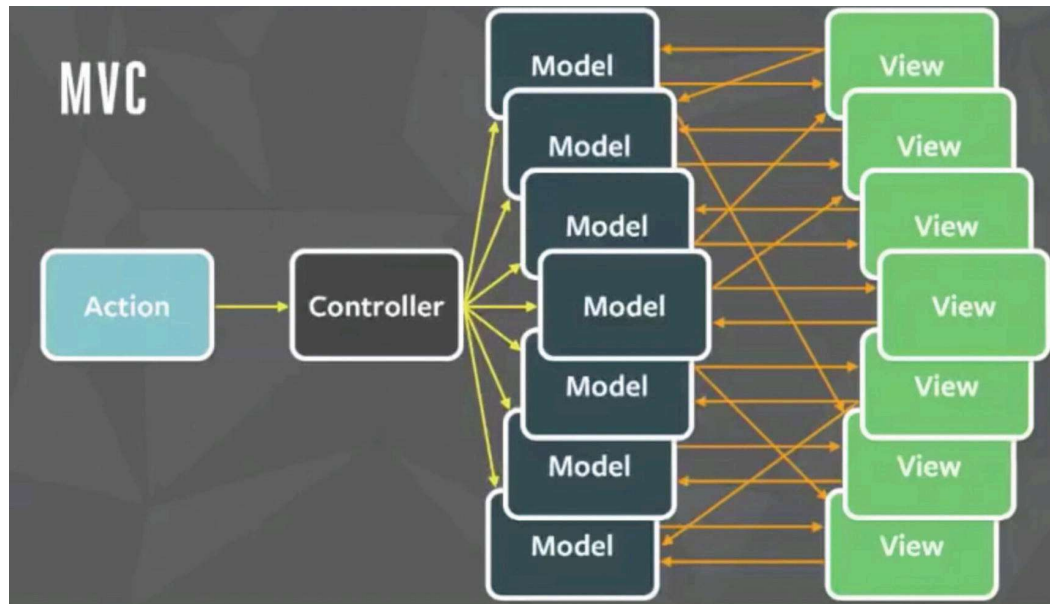


Figure 3-1 Simple MVC pattern

Figure [3-1](#) shows the simplest implementation of MVC, and this works pretty well with small applications. As your application grows, though, so does the demand for new features, and there should be room to accommodate more models and views. Let's look at what happens when our model and view increase in an actual application (Figure [3-2](#)).

**Figure 3-2** MVC pattern for a large application

Wow! That is an explosion of arrows. Welcome to the real world where many models and views interact with each other. A controller triggers another model and this goes on like spaghetti, which often ends up in an infinite loop. The worst part is that it's really difficult to debug code in such a situation, eventually making the system fragile. Well, Facebook faced a similar problem with this pattern and solved it with a new pattern called Flux.

Flux

Flux abjures MVC in favor of a unidirectional data flow. Flux works well because the single directional data flow makes it easy to understand and modify an application as it grows and becomes more complex. Earlier we found that two-way data bindings lead to cascading updates, where change in one data model leads to an update in another data model, making it very difficult to predict what would change as the result of a single user interaction.

Flux applications have three major parts: the dispatcher, the store, and the view (where we use React components). These should not be compared with the model, view, and controller of the MVC pattern (Figure 3-3).

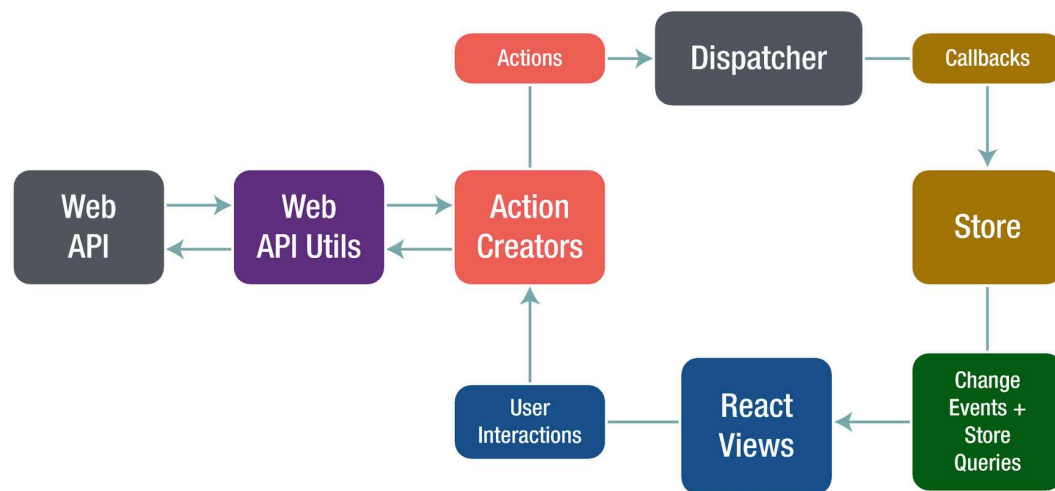


Figure 3-3 React App data flow

Although controllers do exist in a Flux application, these are controller views, where views are found at the top of the hierarchy that retrieve data from the stores and forward these data to their children.

If we look at the Flux architecture, the most important part is the dispatcher, which is a singleton that directs the flow of data and ensures that updates do not cascade (Figure 3-4).



Figure 3-4 Flux data flow

As an application grows, eventually the dispatcher becomes more vital, as it is responsible for managing dependencies between stores by invoking the registered callbacks in a specific order.

When a user interacts with a React view, the view sends an action (usually represented as a JavaScript object with some fields) through the dispatcher, which notifies the various stores that hold the application's data and business logic. When the stores change state, they notify the views that something has updated. This works especially well with React's declarative model, which allows the stores to send updates without specifying how to transition views between states.

The following are some of the key benefits of using Flux:

- It improves data consistency.
- It is easier to pinpoint the bugs.
- You can perform more meaningful unit tests. Because all the states of a module are there in the same place, we can test a module independently.
- It includes predictable code.

With predictable code, great things follow, as shown in Figure [3-5](#).

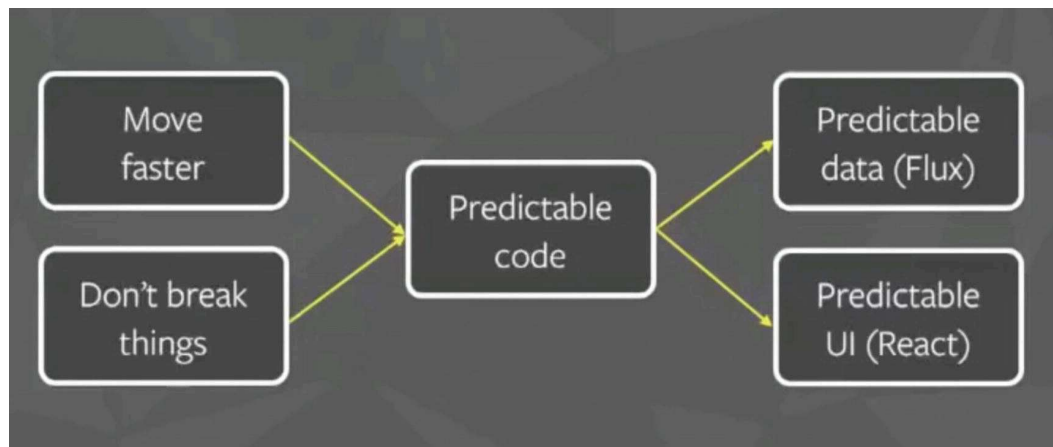


Figure 3-5 Predictable code

Success of Flux

One of Facebook's most popular features was its chat functionality. However, it was extremely buggy and had a high rate of negative user feedback. The new chat system that Facebook implemented is using a Flux pattern that provides a seamless experience. You can have look at example chat code in a Facebook React example at <https://github.com/facebook/flux/tree/master/examples>.

Flux Deep Dive

As we now know what Flux is, let's look into and understand the concepts like dispatcher, store, and action

The Dispatcher

The dispatcher is the central hub that manages all data flow in a Flux application. It is essentially a registry of callbacks into the stores and has no real intelligence of its own; in essence, it is a simple mechanism for distributing the actions to the stores. Each store registers itself and provides a callback. When an action creator provides the dispatcher with a new action, all stores in the application receive the

action via the callbacks in the registry. Dispatcher also acts like a traffic controller. If it gets an action even when the data layer is still processing, it makes sure to run the action. With the dispatcher, you know where your action starts and what changes it makes to the data layer. There are cascading effects that build up in between. You are indeed in full control of your system.

The Need for Dispatcher [`dispatch()` and `waitFor()`]

As an application grows, dependencies across different stores also increase. Imagine, for example, we have a situation where Store A needs Store B to update itself first, so that it can itself know how to update, too. We need the dispatcher to be able to invoke the callback for Store B and finish that callback before moving forward with Store A. To assert this dependence, a store needs to communicate with the dispatcher to first complete the action to update Store B. The dispatcher provides this functionality through the `waitFor()` method.

The `dispatch()` method provides a simple, synchronous iteration through the callbacks, invoking each in turn. When `waitFor()` is encountered within one of the callbacks, execution of that callback stops and `waitFor()` provides us with a new iteration cycle over the dependencies. After the entire set of dependencies has been fulfilled, the original callback then continues to execute.

Further, the `waitFor()` method can be used in different ways for different actions, within the same store's callback. In one case, Store A might need to wait for Store B. In another case, though, it might need to wait for Store C. Using `waitFor()` within the code block that is specific to an action allows us to have fine-grained control of these dependencies.

Problems arise, however, if we have circular dependencies; that is, if Store A needs to wait for Store B, and Store B needs to wait for Store A. This could wind up in an endless loop. The dispatcher now available in the Flux repo protects

against this by throwing an informative error to alert the developer that this problem has occurred. The developer can then create a third store and resolve the circular dependency.

Stores

Stores contain the application state and logic. Their role is somewhat similar to a model in a traditional MVC, but they manage the state of many objects—they do not represent a single record of data like ORM (Object Relational Mapping) models do. More than simply managing a collection of ORM-style objects, stores manage the application state for a particular domain within the application.

As mentioned earlier, a store registers itself with the dispatcher and provides it with a callback. This callback receives the action as a parameter. Within the store's registered callback, a switch statement based on the action's type is used to interpret the action and to provide the proper hooks into the store's internal methods. This allows an action to result in an update to the state of the store via the dispatcher. After the stores are updated, they broadcast an event declaring that their state has changed, so the views can query the new state and update themselves.

Actions

When new data enter the system, whether through a person interacting with the application or through a web API call, those data are packaged into an action—an object literal containing the new fields of data and a specific action type. We often create a library of helper methods called action creators that not only create the action object, but also pass the action to the dispatcher.

Different actions are identified by a type attribute. When all of the stores receive the action, they typically use this attribute to determine if and how they should

respond to it. In a Flux application, both stores and views control themselves; external objects do not act on them. Actions flow into the stores through the callbacks they define and register, not through setter methods.

Letting the stores update themselves eliminates many entanglements typically found in MVC applications, where cascading updates between models can lead to unstable state and make accurate testing very difficult. The objects within a Flux application are highly decoupled, and adhere very strongly to the Law of Demeter, the principle that each object within a system should know as little as possible about the other objects in the system. This results in software that is more maintainable, adaptable, testable, and easier for new engineering team members to understand.

Redux

Now that we have read about Flux, next we discuss another pattern called Redux. Redux can be considered a predecessor to the Flux architecture, and it is also inspired by the functional programming language Elm. Redux was created by Dan Abramov in mid-2015. During that time, the React world was going through major changes and new things were coming every other day. No one, though, could imagine that a small library of just 2 KB would create such a tectonic shift in the way we interact with and create React applications.

Redux was built on top of functional programming concepts. Functional programming by design allows us to write clean and modular code that is easier to test, debug, and maintain. With functional programming, code is in the form of small functions that are isolated in scope and logic, thus making the code reusable. Because small pieces of code are isolated in nature, there is hardly any coupling and these tiny functions can be used as modules in an app. In functional JavaScript you will see pure functions, anonymous functions, and higher order

functions used very often. Redux uses pure functions a lot, so a good understanding of this concept is important.

Pure functions return a value based on arguments passed to them. They do not modify or mutate existing objects, but they return new ones. These functions do not depend on the state from which they are called, but they return only one and the same result for any provided argument. That's why they are very predictable. Because pure functions do not modify any value, they don't have any observable side effects. Redux uses something called reducers, which are pure functions. We will learn in detail about reducers and other Redux code concepts in the next section.

Redux Core Concepts

Redux has three core pillars: action, store, and reducers (Figure [3-6](#)). These words might sound complicated, but they are actually very simple.

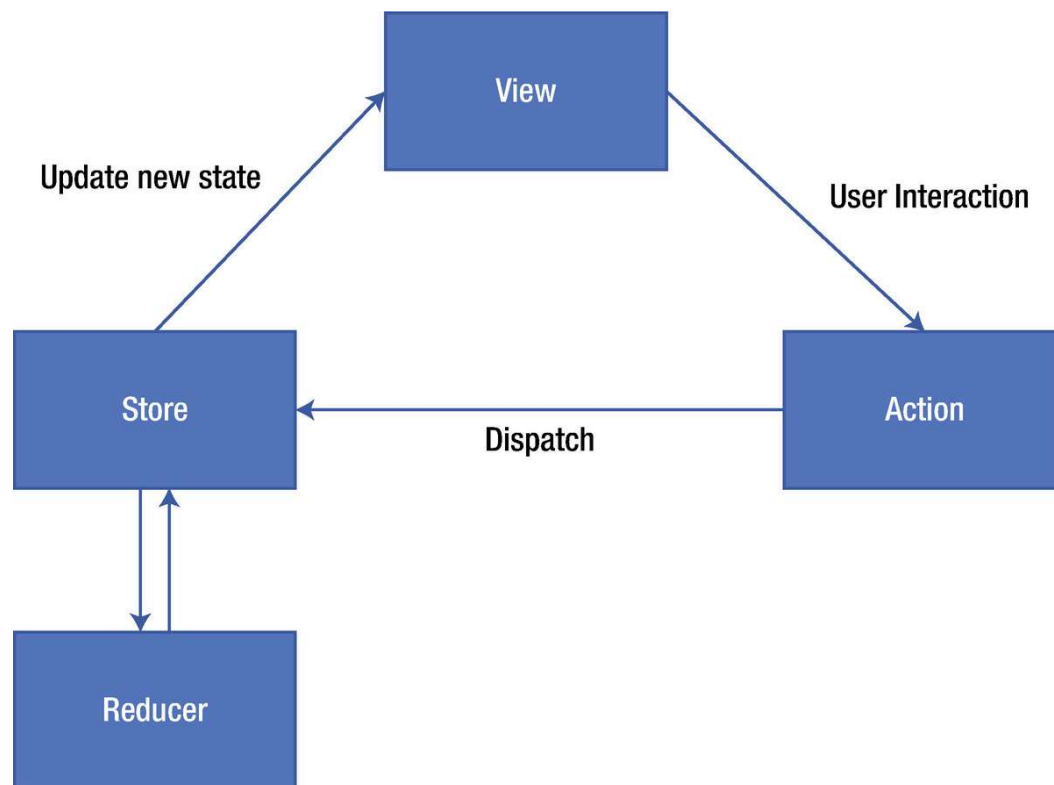


Figure 3-6 Redux data flow

Action

Actions are events that send data from the application (user interactions, API calls, form submissions, etc.) to the store. The store always gets the information from actions. Internal actions are simple JavaScript objects that have a `type` property (usually constant), describing the type of action and payload of information being sent to the store. To send them to the store we use `store.dispatch()`.

Action creators, as the name suggests, are the functions that create actions. It is easy to conflate the terms *action* and *action creator*, so do your best to use the proper term. To call these action creator functions anywhere in the app we use `dispatch`. As mentioned earlier, the `dispatch()` function can be accessed directly from the store as `store.dispatch()`, but more likely you'll access it using a

helper like react-redux's `connect()` method. You can use `bindActionCreators()` to automatically bind many action creators to a `dispatch()` function.

Note Action creators can also be asynchronous and have side effects. This is an advanced topic, so we don't need to go in-depth right now.

Reducer

Reducers specify how the application's state changes in response to actions sent to the store. Remember that actions only describe what has happened, but do not tell anything about state change in the application. Understanding how reducers work is important in an application using Redux because they are responsible for most of the work. Let's describe this with a simple example.

```
function appAuth(state, action) {  
  return Object.assign({}, state, {  
    authType: action.payload  
  });  
}
```

This is a very simple reducer that takes the current state and an action as arguments and then returns the next state. In the case of complex applications, we will be using the `combineReducers()` utility that is provided by Redux. It combines all the reducers of the app into a single index reducer. Every reducer is responsible for its own part of the app's state, and the state parameter is different for every reducer. The `combineReducers()` utility makes the file structure much easier to maintain.

Store

Thus far we have learned that actions represent what happened and the reducers update the state according to those actions. The store is the object that brings them all together.

The store holds the application state and provides a few helper methods to access the state, dispatch actions, and register listeners. The entire state is represented by a single store. Any action returns a new state via reducers. The following are few helper methods:

- `getState()`: Allows access to state.
- `dispatch(action)`: Allows state to be updated.
- `subscribe(listener)`: Registers listeners.
- `replaceReducer(nextReducer)`: Replaces the reducer currently used by the store to calculate the state.

Redux with React Native

To understand how Redux works with React Native, let's create a simple Todo application in React Native including how Redux makes things simple for us. To proceed, use the source code available for this chapter. Inside the project directory, run `yarn install` to install two new packages.

```
redux
react-redux
```

The Redux module is required so that you can use Redux with your application. React-redux is going to help you connect your React native app to Redux once you have both of these installed. Figure [3-7](#) shows our application folder structure.

- ▶ .expo
- ▶ assets
- ▶ node_modules

- ◀ src
 - ▶ actions
 - ▶ components
 - ▶ containers
 - ▶ reducers
 - ▶ store
- JS TodoApp.js

6 .babelrc

📄 .gitignore

{ } .watchmanconfig

JS App.js

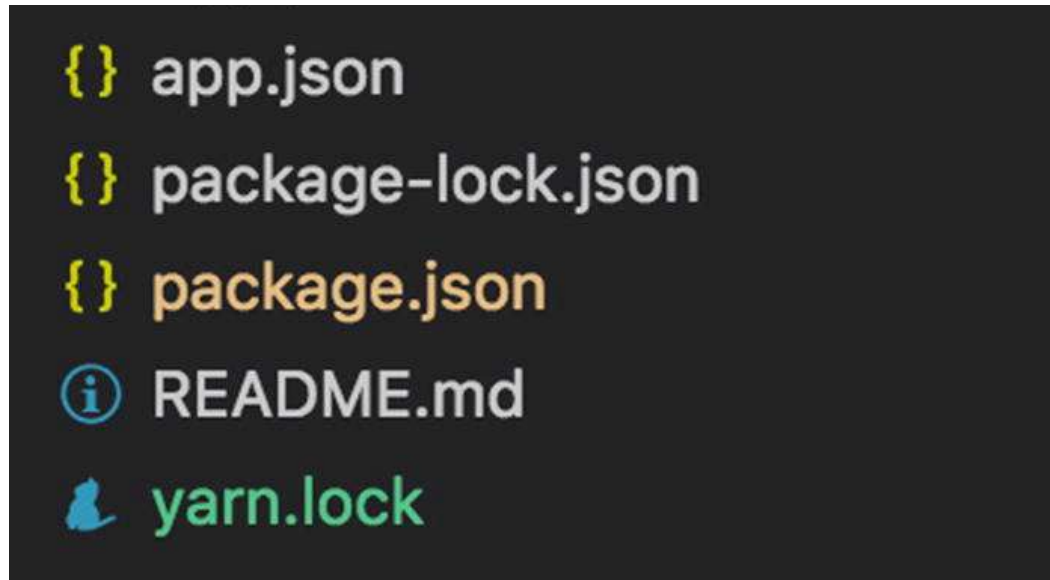


Figure 3-7 List of project folder structure

You would have to create all these folders: `components`, `containers`, `reducers`, `store`, and a `TodoApp.js` file. Within these folders we would have more JavaScript files reside inside our action, stores, reducers, and components. This way our code stays modularized and the logic remains isolated. Here, the Redux part is managed under the action, reducer, and the store folder, but we would need components that will use them.

Hence, we have two folders here: `components`, which consists of plain dumb components, which are the presentational components of the app having no idea that Redux exists or not in the app. Second, we have smart components that interact with Redux, and they reside in the `containers` folder.

First, let's create a store. Inside the `store` folder create an `index.js` file and paste the following code:

```
import { createStore } from 'redux'
import rootReducer from '../reducers'
```

```
export default store = createStore(rootReducer)
```

Here, we have imported something called `createStore` from `redux`. Here we are combining all our reducers with `rootReducer` and exporting the same. Soon you will see how we have created two reducers that we plan to use with our store using `rootReducer`.

Next, this store is imported into the application by adding the following code in our `App.js`.

```
import store from './src/store'
import { Provider } from 'react-redux'
export default class App extends React.Component {
  render() {
    return (
      <Provider store={store}>
        <TodoApp />
      </Provider>
    );
  }
}
```

Here, we have imported our store and also used something called `Provider` from `react-redux`. Once we pass our `Provider` and store within that, it can be accessed anywhere in `TodoApp` no matter how many levels deep it is. Great! With this our store is set up.

Although our store is setup, we require some UI components. If you look at the `containers` folder, we have an `addTodo` component, which is a simple `TextInput` that will be used to create a new `todo`. Therefore, on this text input field there will be some action that will trigger it to create a new `todo`.

Before we create our action and reducer, we must think about what states we can have in this application. As per our design, there should be two states: one `Todo` and one for visibility. Therefore, in the `reducers` folder, you will find two files: one for the `todo` reducer and other for the `visibilityFilter` reducer .

```
const visibilityFilter = (state = "SHOW_ALL", action) => {  
  return state  
}
```

Here the visibility filter reducer takes the state `SHOW_ALL` and based on the action, it gives the expected outcome and returns the state. In this case, it is responsible to show all the `todo` records. Next, let's look at the `todo` reducer :

```
const todos = (state = [], action) => {  
  switch (action.type) {  
    case 'ADD_TODO':  
      return [  
        ...state, {  
          id: action.id,  
          text: action.text,  
          completed: false  
        }  
      ]  
    case 'TOGGLE_TODO':  
      return state.map(todo =>  
        (todo.id === action.id)  
          ? { ...todo, completed: !todo.completed } :  
          todo)  
    default:  
      return state  
  }  
}  
export default todos
```

Here we have two actions—`ADD_TODO` and `TOGGLE_TODO`—that are responsible for adding a new record in the list and marking a record complete using their respective actions. We are able to determine the type of actions using `action.type`.

At this point, both the reducers are independent, so we need a way to combine them, which you can find in the third file in the `reducers` folder, `index.js`. Let's look at the code inside `reducers/index.js`.

```
import { combineReducers } from 'redux'
import todos from './todos'
import visibilityFilter from './visibilityFilter'
export default combineReducers({
  todos,
  visibilityFilter
})
```

Here we are using something called `CombineReducers` from `redux`. This helps is keep the logical part separate but use it such in a way that we have only one reducer.

Working with the Components

Now that we have seen our reducers, let's see how all this so far works with our components. Open the `containers/AddTodo.js` file.

```
import { connect } from 'react-redux '
import { addTodo } from '../actions'
class AddTodo extends Component {
  state = {
    text: ' '
  }
  addTodo = (text) => {
    // redux store
```

```

    this.props.dispatch(addTodo(text))
    this.setState({ text: '' })
  }
  render() {
    return (
      <View style={{ flexDirection: 'row', marginHorizontal: 20 }}>
        <TextInput
          onChangeText={({text) => this.setState({ text })}
          value={this.state.text}
          placeholder="E.g. Create New Video"
          style={{ borderWidth: 1, borderColor: '#f2f2e1', backgroundColor: '#eaeaea', height: 50, flex: 1, padding: 5 }}
        />
        <TouchableOpacity onPress={() => this.addTodo(this.state.text)}>
          <View style={{ height: 50, backgroundColor: '#eaeaea', alignItems: 'center', justifyContent: 'center' }}>
            <Icons name="md-add" size={30} style={{ color: '#de9595', padding: 10 }} />
          </View>
        </TouchableOpacity>
      </View>
    );
  }
}
export default connect()(AddTodo);

```

Here, we first have to set up the initial local state:

```

state = {
  text: ''
}

```

This is just an empty string for the text input to stay empty. Next, we have to update the text from text input when a user types. This is done using `onChangeText` where the state is updated with the text entered by the user.

```

onChangeText={({text) => this.setState({ text })}

```

```
value={this.state.text}
```

Once the user submits the `todo` it must be updated to our store. For this, we use a helper method, `connect`, from `react-redux`.

```
import { connect } from 'react-redux'
```

In addition, the `connect` helper method should be passed with the component that is going to get connected to the Redux store. In this case, use `AddTodo`:

```
export default connect()(AddTodo);
```

We also have to import the action we plan to use; in this case, it is `addTodo`:

```
import { addTodo } from '../actions'
```

Great! Now that we have connected our store to the component, let's trigger it `onPress` to add the `todo` in a list:

```
<TouchableOpacity onPress={() => this.addTodo(this.state.text)}>
```

Because this component is connected to the Redux store, we can dispatch the action to the respective store:

```
addTodo = (text) => {  
  this.props.dispatch(addTodo(text))  
  this.setState({ text: '' })  
}
```

This will update the `todo` list and also the text input state with an empty string so that new values can be added later.

To display the data, we use a dumb component whose only purpose is to display the to-do list. This component has nothing to do with Redux. You can find this dumb component inside `component/ToDoList.js`

Finally, when we run our application, we will find the result shown in Figure [3-8](#) in the simulator.

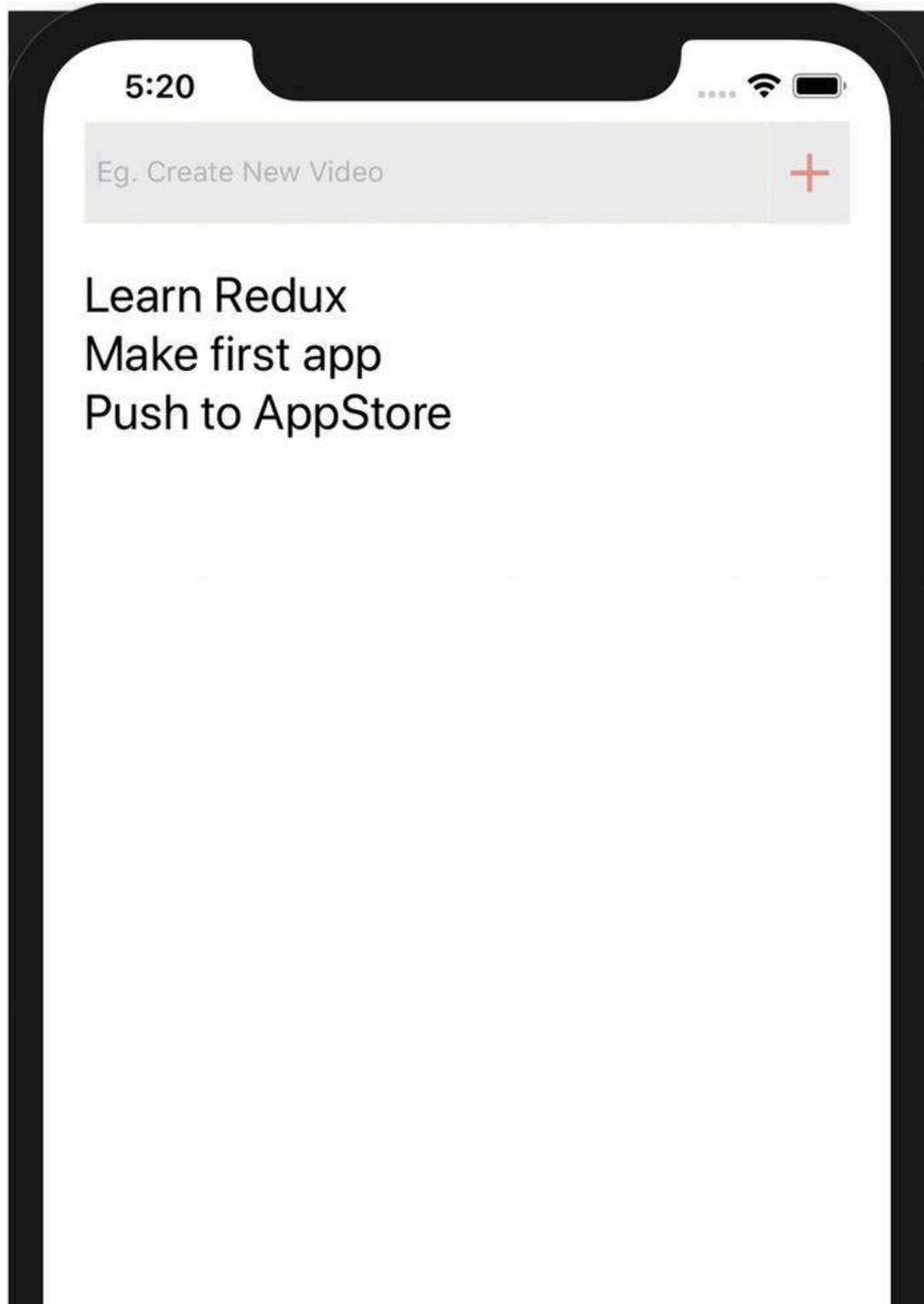


Figure 3-8 Showing the to-do list on an iPhone

As you saw, there is some work involved in using Redux along with your application, and as with any new piece of technology or new pattern, developers should always ask this: Why should I use it in the first place?”

It might be a little complicated to understand and implement something with Redux, but once you understand the fundamentals, it provides many advantages, including these:

- *Expected outcomes:* With Redux there is no confusion about where to locate our one source of truth; that will always be the store.
- *Maintainability and organization of code:* With a strict structure in place and predictable outcomes, maintaining the code becomes easier. Redux is also particular about how the code should be organized, and this becomes pivotal in maintaining the source code as an application becomes large.
- *Tools:* With developer tools, developers can track what’s happening in the application in real time.
- *Community:* Redux is not something that has just appeared; it has indeed passed the test of time. The community is flourishing, and you can easily get support and regular updates for the library.
- *Ease of testing:* Redux functions by design are small, pure, and isolated, which makes them perfect candidates to for which to write tests. Redux apps automatically make testing easy for the application.

Summary

In this chapter you learned about the Flux pattern, how it differs, and how it solves a fundamental problem differently from the traditional MVC pattern. We also looked closer at Flux core concepts. Next, you learned about the successor of Flux, Redux, its core concepts, benefits, and how to use it with React Native appli-

cations, which will be useful in real-world applications and in the upcoming chapters. Chapter [4](#) covers how to create UIs and navigation in React Native apps. Finally, you learn how to use animation in your views.